Lesson 2 Color Tracking

1. Program Logic

With camera vision module, ArmPi can recognize different colors visually.

First, program ArmPi to recognize colors with Lab color space. Convert the RGB color space to Lab, image binarization, and then perform operations such as expansion and corrosion to obtain an outline containing only the target color. Use circles to frame the color outline to realize object color recognition.

Secondly, judge whether the object moves after recognition. Obtain the center coordinates of the object, transform the coordinates into reality, and then compare the last coordinates by drawing the center point to determine whether it is a movement. If the object is still moving, the robotic arm will track it. When the object stops moving for few seconds, the robotic arm will grip it.

Gripping process: The arm moves to a height of 7cm from the target position. Open the gripper, after calculating rotate angle, lower to 2cm from the target position and then close to grip the block.

Last step is to raise the arm, sort and place objects of different colors, and then return to the initial position.

The source code of the program is located in：/home/pi/ArmPi/Functions/ColorTracking.p

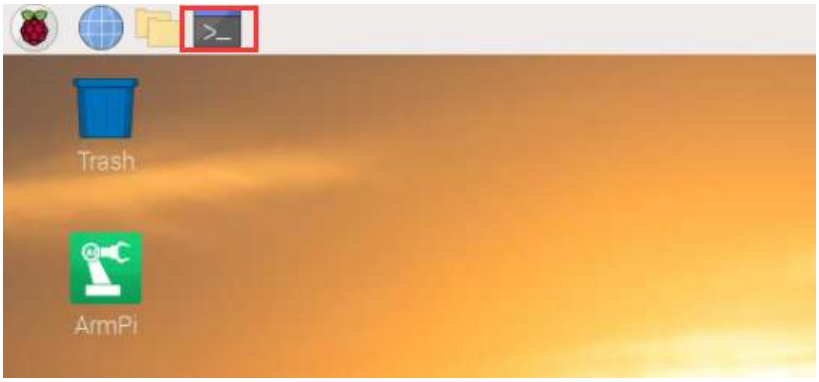
```
frame_lab = cv2.cvtColor(frame_gb, cv2.COLOR_BGR2LAB)  # convert the image to LAB space

area_max = 0
areaMaxContour = 0
if not start_pick_up:
    for i in color_range:
        if i in __target_color:
            detect_color = i
            frame_mask = cv2.inRange(frame_lab, color_range[detect_color][0], color_range[detect_color][1])  # mathematical operation on the original image and mask
            opened = cv2.morphologyEx(frame_mask, cv2.MORPH_OPEN, np.ones((6, 6), np.uint8))  # Opening (morphology)
            closed = cv2.morphologyEx(opened, cv2.MORPH_CLOSE, np.ones((6, 6), np.uint8))  # Closing (morphology)
            contours = cv2.findContours(closed, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)[-2]  # find countour
            areaMaxContour, area_max = getAreaMaxContour(contours)  # find the maximum countour
    if area_max > 2500:  # find the maximum area
        rect = cv2.minAreaRect(areaMaxContour)
        box = np.int0(cv2.boxPoints(rect))

        roi = getROI(box) # get roi zone
        get_roi = True

        img_centerx, img_centery = getCenter(rect, roi, size, square_length)  # get the center coordinates of block
        world_x, world_y = convertCoordinate(img_centerx, img_centery, size) # convert to world coordinates


        cv2.drawContours(img, [box], -1, range_rgb[detect_color], 2)
        cv2.putText(img, '(' + str(world_x) + ',' + str(world_y) + ')', (min(box[0, 0], box[2, 0]), box[2, 1] - 10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, range_rgb[detect_color], 1) # draw center position
        distance = math.sqrt(pow(world_x - last_x, 2) + pow(world_y - last_y, 2)) # compare the last coordinate to determine whether to move
        last_x, last_y = world_x, world_y
        track = True
        #print(count,distance)
        # cumulative judgment
        if action_finish:
            if distance < 0.3:
                center_list.extend((world_x, world_y))
                count += 1
                if start_count_t1:
                    start_count_t1 = False
                    t1 = time.time()
                if time.time() - t1 > 1.5:
                    rotation_angle = rect[2]
                    start_count_t1 = True
                    world_X, world_Y = np.mean(np.array(center_list).reshape(count, 2), axis=0)
                    count = 0
                    center_list = []
                    start_pick_up = True
            else:
                t1 = time.time()
                start_count_t1 = True
                count = 0
                center_list = []
return img
```

2. Operation Steps

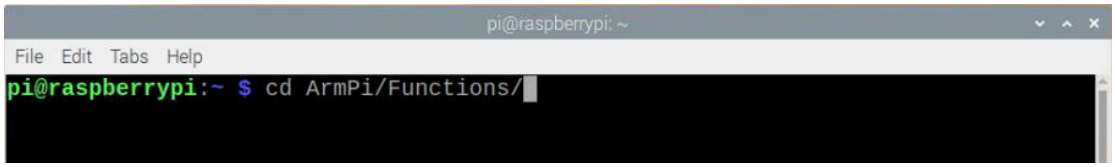
Pay attention to the text format in the input of instructions.

1) Turn on robotic arm and connect to Raspberry Pi desktop with VNC.

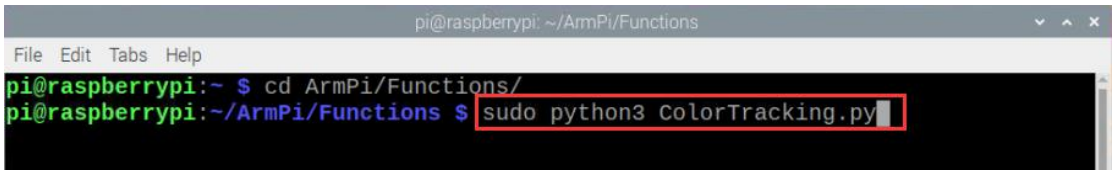
2) Click  or press “Ctrl+Alt+T” to open the LX terminal.



3) Enter “cd ArmPi/Functions/” command, and then press “Enter” to come to the category of games programmings.



4) Enter “sudo python3 ColorTracking.py”, then press “Enter” to start the game.

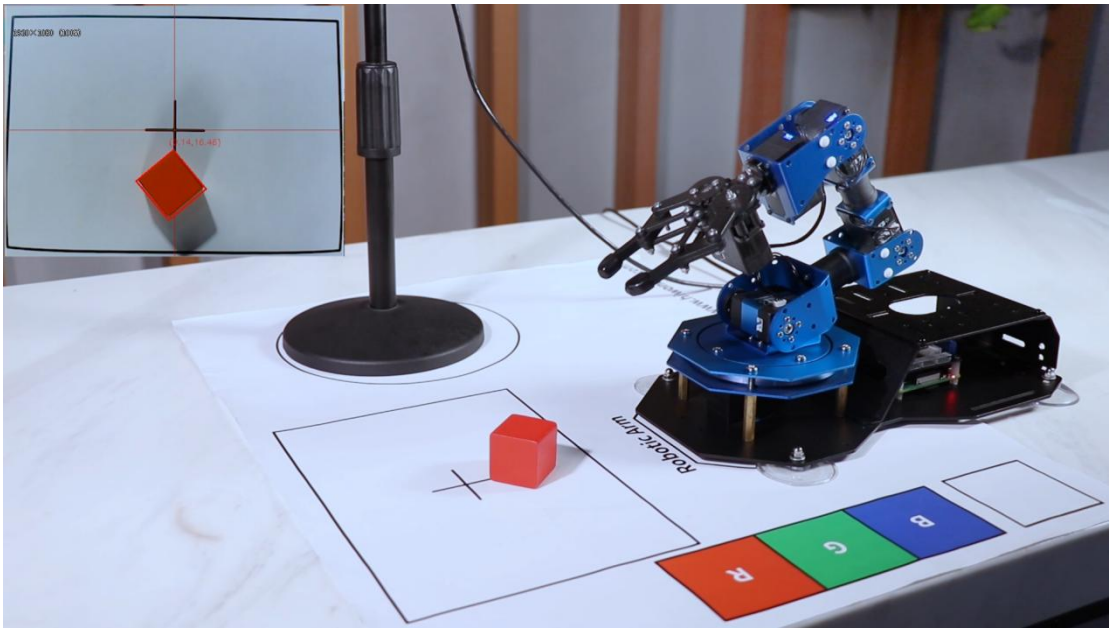


5) If you want to exit the game programming, press “Ctrl+C” in the LX terminal interface. If the exit fails, you can press it multiple times.

3. Project Outcome

The program's default recognition and tracking color is red. If you want to change to green or blue, please refer to below “4.1 Modify Program Default Recognition Color”. Please note that do not move too fast when you hold the block and should meet the range of camera recognition.

Place the red block in the vision recognition area. When the block is recognized. the buzzer on the Raspberry Pi expansion board will emit “beep”, two RGB lights will be on red and the screen returned by the camera will circle the box.



Hold the red block to move slowly and robotic arm will move to chase it. When the object stops moving over 1.5s, ArmPi will run to grip it.

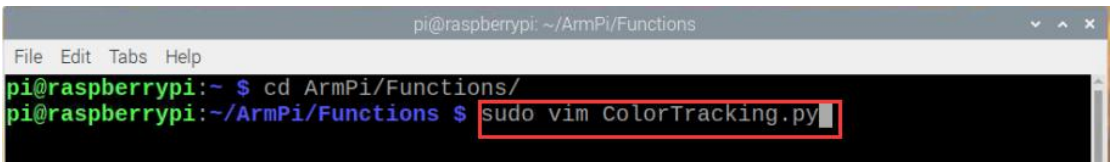
4. Function Extension

4.1 Modify Program Default Recognition Color

The default recognition color can be modified in the APP by clicking button directly. If you want to use the command line to execute the program, you may need to modify the program.

Step1: Enter command “cd ArmPi/Functions/” to the directory where the game program is located.

Step2: Enter command “sudo vim ColorTracking.py” to go into the game program through vi editor.



Step3: Input “380” and press “shfit+g” to the line for modification.

```
377 if __name__ == '__main__':
378     init()
379     start()
380     _target_color = ('red', )
381     my_camera = Camera.Camera()
382     my_camera.camera_open()
383     while True:
384         img = my_camera.frame
385         if img is not None:
386             frame = img.copy()
387             Frame = run(frame)
388             cv2.imshow('Frame', Frame)
389             key = cv2.waitKey(1)
390             if key == 27:
```

Step4: Press “i” to enter the editing mode, then modify red in _target_color = (‘red’,) to blue or green.

Step5: Press “Esc” to enter last line mode. Input “wq” to save and exit.

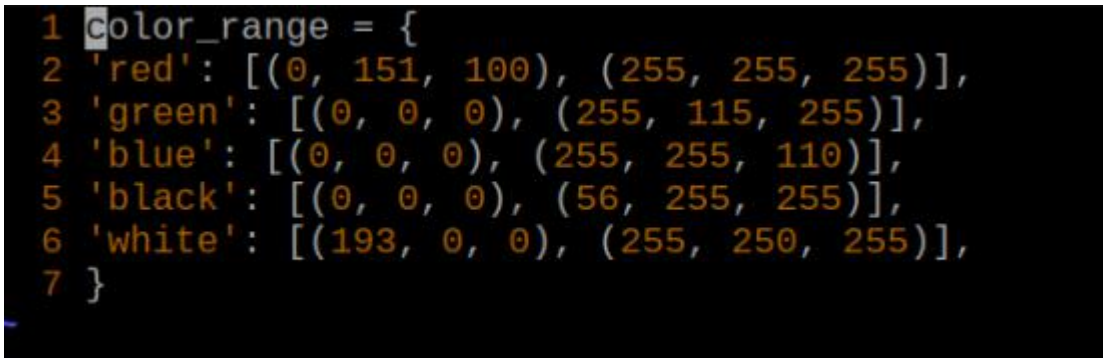
```
388             cv2.imshow('Frame', Frame)
389             key = cv2.waitKey(1)
390             if key == 27:
391                 break
392     my_camera.camera_close()
393     cv2.destroyAllWindows()
~
~
~
~
~
:wq
```

4.2 Add Recognition Color

In addition to the built-in recognized color, you can set other recognized colors in the programming.

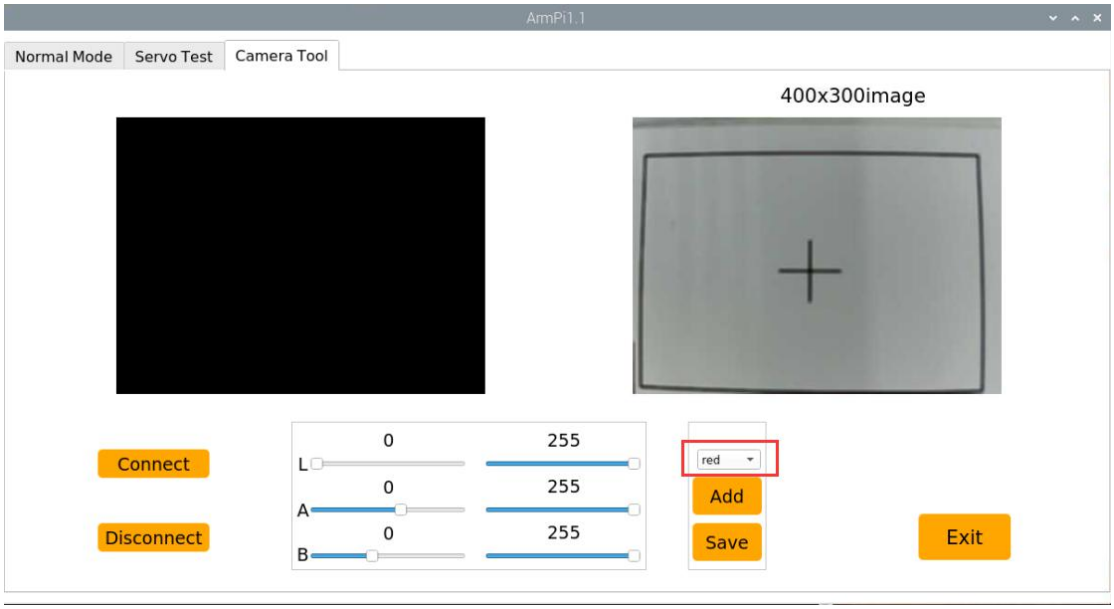
1) Open VNC, input command “sudo vim ArmPi/LABConfig.py” to open Lab color setting document.

It is recommended to use screenshot to record the initial value (it will be better to to change the value back after the modification is completed later for the outcome of the game.

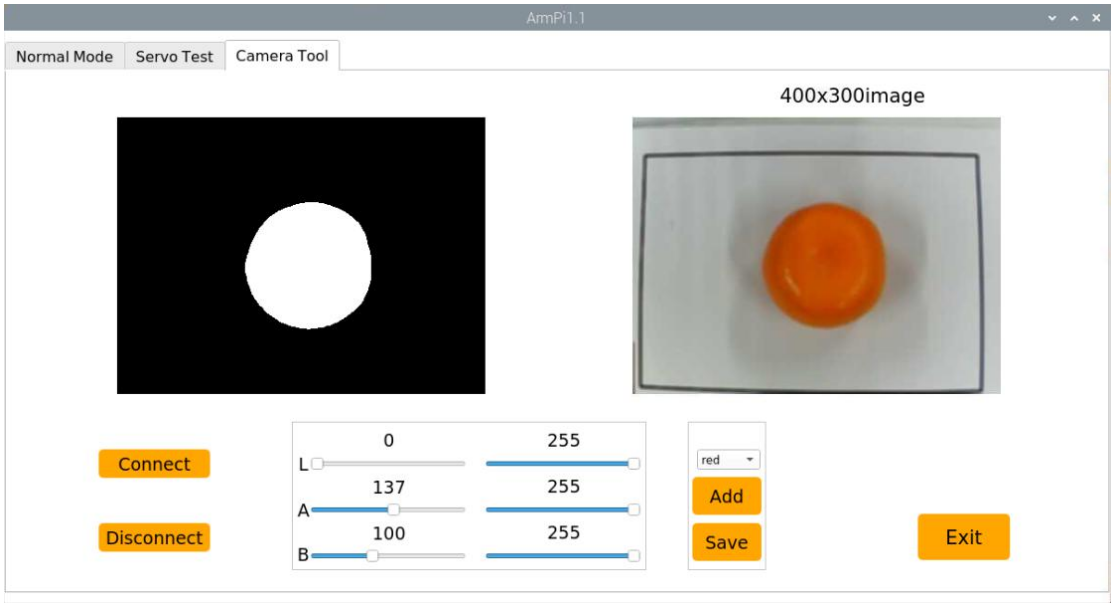


2) Click the ArmP icon in the system desktop. Choose “Run” in the pop-up window.

3) Click “Connect” button in the lower left corner. When the interface display the camera return screen, the connection is successful. Select "red" in the right box first because the default recognition color of the programming is red). Then drag the corresponding sliders of L, A, and B until the color area to be recognized in the left screen becomes white and other areas become black.



4) Point the camera at the color you want to recognize. For example, if you want to recognize orange, you can put the orange ball in the camera's field of view. Adjust the corresponding sliders of L, A, and B until the orange part of the left screen becomes white and other colors become black, and then click " Save" button to keep the modified data.

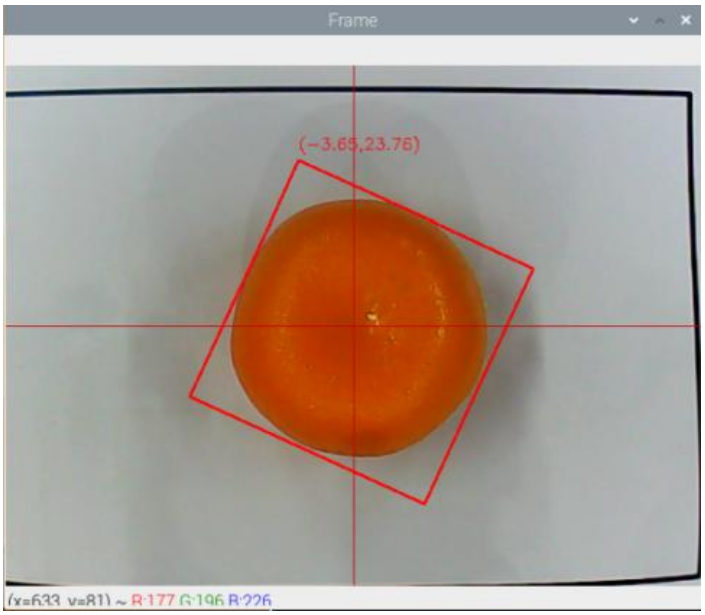


5) After the modification is completed, we can check whether the modified data was successfully written in. Enter the command again “sudo vim ArmPi/LABConfig.py” to check the color setting parameters.

6) Check the data in red frame. If the edited value was written in the program, press “Esc” and “:wq” to save and exit.

```
color_range = {
'red': [(0, 137, 100), (255, 255, 255)],
'green': [(0, 0, 0), (255, 115, 255)],
'blue': [(0, 0, 0), (255, 255, 110)],
'black': [(0, 0, 0), (56, 255, 255)],
'white': [(193, 0, 0), (255, 250, 255)],
}
```

7) According to the previous study, start the game again. Put the orange ball in front of the camera and you can see that the recognition is successful.



8) If you want to modify to other colors, please operate as the above steps.